Sesión 8 — Laboratorio

Programación Declarativa
3º del Grado en Ingeniería Informática
Listas y cadenas por comprensión: cifrados César y Vigenère

Nota

Abre GHCi y carga tus definiciones con `:load fichero.hs` (o `:l fichero.hs`). Recuerda que puedes recargar tras editar con `:reload` (o `:r`). Comprueba los tipos de tus funciones con `:type` (`:t`) antes de probarlas.
Los ejercicios marcados con ★ son los recomendados para completar durante la sesión si el tiempo es limitado.

Parte A — Calentamiento con listas por comprensión

Ejercicio 1.

Define `multiplosDe :: Int -> Int -> [Int]` que, dados `n` y un límite, devuelva todos los múltiplos de `n` hasta ese límite (incluido), usando una lista por comprensión.

```
ghci> multiplosDe 3 20
[3,6,9,12,15,18]
```

Ejercicio 2.

Define `sumaCuadradosImpares :: Int -> Int` que sume los cuadrados de todos los números impares entre 1 y `n` (incluido).

```
ghci> sumaCuadradosImpares 10
165
```

Ejercicio 3.

Define `triangulosRectangulos :: Int -> [(Int,Int,Int)]` que devuelva las ternas (a, b, c) con $1 \le a \le b \le c \le n$ tales que $a^2 + b^2 = c^2$ y además $a + b + c$ sea un número par.

```
ghci> triangulosRectangulos 20
[(3,4,5),(6,8,10),(5,12,13),(9,12,15),
 (8,15,17),(12,16,20)]
```

Ejercicio 4. ★

Define `productoCartesiano :: [a] -> [b] -> [(a,b)]` que devuelva todos los pares posibles formados por un elemento de la primera lista y un elemento de la segunda, usando dos generadores.

```
ghci> productoCartesiano [1,2] "ab"
[(1,'a'),(1,'b'),(2,'a'),(2,'b')]
```

Parte B — Cadenas por comprensión: la base de los cifrados

Los ejercicios de cifrado necesitan convertir entre letras y números. Para simplificar, trabajaremos siempre con mensajes en **minúsculas**. El módulo `Data.Char` (ya usado en la sesión anterior para `toUpper`) ofrece también estas dos funciones:

```
ord :: Char -> Int
-- codigo numerico (Unicode) de un caracter
chr :: Int -> Char
-- caracter correspondiente a un codigo numerico (inversa de ord)
isLower :: Char -> Bool
-- comprueba si un caracter es una letra minuscula
```

```
ghci> import Data.Char
ghci> ord 'a'
97
ghci> chr 98
'b'
ghci> isLower 'a'
True
```

Ejercicio 5. ★

Define `posicionEnAlfabeto :: Char -> Int` que devuelva la posición de una letra **minúscula** dentro del alfabeto (`'a'` es 0, `'b'` es 1, …, `'z'` es 25), usando `ord`.

```
ghci> posicionEnAlfabeto 'a'
0
ghci> posicionEnAlfabeto 'd'
3
```

Ejercicio 6. ★

Define `letraEnPosicion :: Int -> Char` que sea la función inversa de la anterior: dada una posición (no necesariamente entre 0 y 25), devuelva la letra minúscula correspondiente, usando `chr` y el operador `mod` para que la posición "dé la vuelta" al alfabeto.

```
ghci> letraEnPosicion 3
'd'
ghci> letraEnPosicion 26
'a'
ghci> letraEnPosicion (-1)
'z'
```

Ejercicio 7. ★

Define `desplazaLetra :: Int -> Char -> Char` que desplace una letra **minúscula** `n` posiciones en el alfabeto, circularmente, dejando **intactos** los caracteres que no sean letras (espacios, signos de puntuación, …). Usa `posicionEnAlfabeto` y `letraEnPosicion` como ayuda.

```
ghci> desplazaLetra 3 'a'
'd'
ghci> desplazaLetra 3 'x'
'a'
ghci> desplazaLetra 3 ','
','
```

Parte C — Cifrado César

Ejercicio 8.

Define `cifradoCesar :: Int -> String -> String` que cifre un mensaje desplazando cada letra `n` posiciones, usando una cadena por comprensión y `desplazaLetra`.

```
ghci> cifradoCesar 3 "hola, mundo!"
"krod, pxqgr!"
```

Ejercicio 9.

Define `descifradoCesar :: Int -> String -> String` que descifre un mensaje cifrado con `cifradoCesar n`, reutilizando `cifradoCesar` con el desplazamiento opuesto.

```
ghci> descifradoCesar 3 "krod, pxqgr!"
"hola, mundo!"
```

Ejercicio 10.

Define `frecuenciaLetras :: String -> [(Char,Int)]` que cuente cuántas veces aparece cada letra (ignorando los caracteres que no son letras) en un texto en minúsculas, devolviendo una lista de pares letra/número de ocurrencias solo para las letras que aparecen al menos una vez.

```
ghci> frecuenciaLetras "abracadabra"
[('a',5),('b',2),('r',2),('c',1),('d',1)]
```

Pista: usa `isLower` y la función `count` vista en la Sesión 5 para contar ocurrencias.

Ejercicio 11.

Define `descifradoFuerzaBruta :: String -> [String]` que pruebe los 26 desplazamientos posibles (`0` a `25`) sobre un mensaje cifrado con César, devolviendo la lista de los 26 resultados, para poder inspeccionarlos visualmente y encontrar el que tenga sentido.

```
ghci> take 4 (descifradoFuerzaBruta "krod, pxqgr!")
["krod, pxqgr!","jqnc, owpfq!",
 "ipmb, nvoep!","hola, mundo!"]
```

Parte D — Cifrado Vigenère

Ejercicio 12.

Define `repiteClave :: String -> Int -> String` que repita una clave hasta alcanzar (al menos) una longitud dada, usando `cycle` y `take`.

```
ghci> repiteClave "sol" 7
"solsols"
```

Ejercicio 13. ★

Define `cifradoVigenere :: String -> String -> String` que cifre un mensaje usando una clave: cada letra del mensaje se desplaza según la posición en el alfabeto de la letra correspondiente de la clave (repetida tantas veces como sea necesario). Los caracteres no alfabéticos del mensaje no se desplazan, y **no consumen** una letra de la clave.

```
ghci> cifradoVigenere "sol" "hola, mundo!"
"zcws, affrz!"
```

Pista: primero calcula la lista de desplazamientos (las posiciones de cada letra de la clave repetida) y combínala con el mensaje usando usando una función recursiva.

Ejercicio 14. ★

Define `descifradoVigenere :: String -> String -> String` que descifre un mensaje cifrado con `cifradoVigenere`, usando los desplazamientos opuestos.

```
ghci> descifradoVigenere "sol" "zcws, affrz!"
"hola, mundo!"
```